What IronClaude is

IronClaude is a set of lessons learned about using AI well, wrapped into skills, so that long runs give reliable, repeatable output.

> **Skill:** a packaged set of instructions Claude loads for one kind of work, such as planning, debugging or review. You start most of them with a slash command, for example `/sc:workflow`.

The routing guide, `HOW-WE-WORK.md`, tells you which skill or command to use for each kind of work, and in what order. `superclaude install` copies it to `~/.claude/HOW-WE-WORK.md`. Keep it open while you work: this deck explains the *why*, and the guide holds the exact commands and flags.

> *As it was put in the onboarding meeting: think of the skills as the basic instructions an AI needs to give reliable, repeatable output over a long run.*

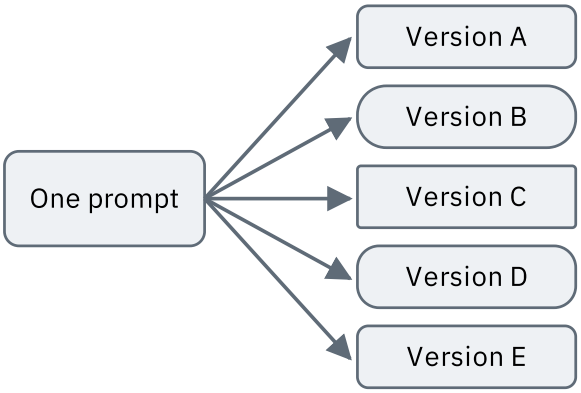
Where anything you hear or read differs from the guide, the guide is the authority.

The problem: consistency

Ask an agent to do the same thing five times and you get five different versions. Each one roughly does what you asked, in a slightly different way, and each one skips something different.

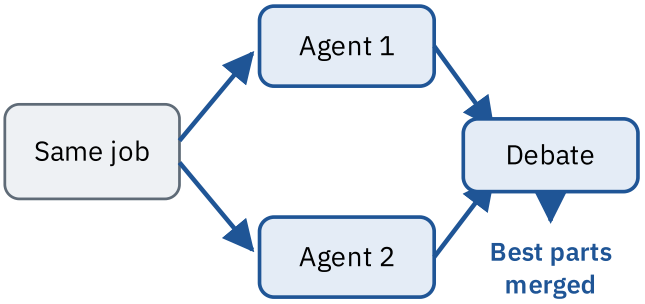
> *In the onboarding meeting the view was that, only a few model generations ago, the best models followed instructions about 70% of the time. That is an opinion from the meeting, not a measured figure, but it explains the design: the heavier skills were built for it, so they add gates, run important work twice, and make agents debate the results.*

Asked once, five times



Five different answers; each misses something different.

Two agents, then a debate



The compare step catches what one run missed.

One prompt fans out to five different results; two agents plus a debate keep the best parts.

How we work as a team

Tokens are cheap; side quests are expensive

Tokens are the units AI usage is measured and paid in.

> *The advice in the onboarding meeting: do not get pulled into optimizing the harness (the IronClaude and Claude Code setup itself). Buying more capacity costs less than the downtime of a rabbit hole.*

Drive your work through the commands

Pick the level and the command from the guide before you start (slide 5).

> *In the meeting, the speaker said about 90% of their own recent work runs through these commands, and the aim was stated plainly: if the commands keep proving to be overkill for your work, the skills should change to fit it.*

Important work gets checked twice

The heavier skills have two agents do the same job, often on different models, and then compare or debate the results. `/sc:adversarial` runs that debate; `/sc:reflect` checks finished work against its plan, and brings in several reviewers on different models when it is not confident or you ask for a deep check.

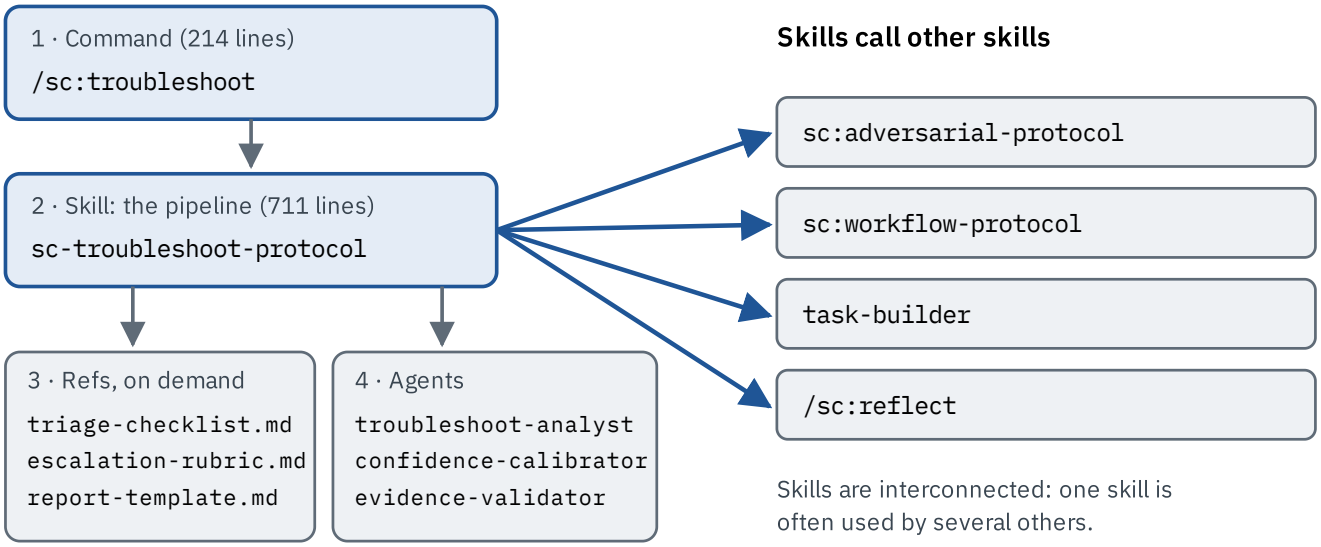
Some of the guide's rules (section 4)

- Write it down: plans, research and findings are saved to files.
- Let a skill finish; do not repeat its own checks.
- Never turn off a quality check; if one seems wrong or too slow, raise it with your lead.
- Read a plan before you run it.
- Do only what was asked; suggest anything else.

Anatomy of a command

A command is a thin front door. It tells you and the agent how to use the skill and where the skill lives. The skill holds the actual pipeline.

> **Ref:** an extra instruction file inside a skill's `refs/` folder. **Progressive disclosure:** the skill loads each ref only at the step that needs it, so instructions you do not need never take up room in the conversation.



Command, skill, refs (templates such as `report-template.md` are ref files) and agents, with the skills `/sc:troubleshoot` calls.

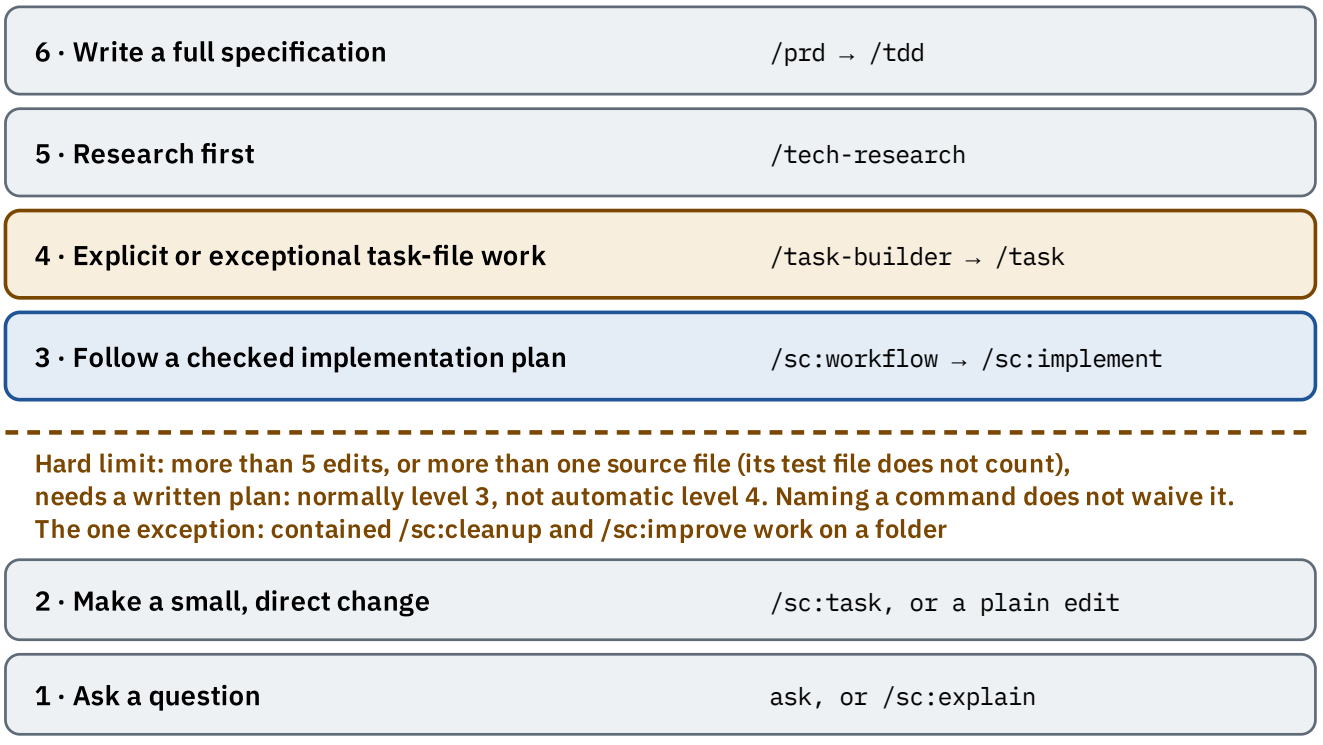
> *From the meeting: the skills are interconnected, so before you change one, know that several others probably use it.*

Pick your level

The six levels set how much process a change gets. Match the outcome you want first, and any command you named or plan that already exists. Most planned implementation uses level 3; level 4 is only for an explicit or exceptional need for its task-file machinery.

Hard limit: if the work needs more than 5 edits or touches more than one source file (its matching test file does not count), it must go through a written plan: normally level 3, not automatic level 4. The one exception is contained `/sc:cleanup` and `/sc:improve` work on a folder (guide 3.7): commit first and review `git diff` afterwards. Naming a command directly does not waive the written plan.

> **Spec:** a short written description of what changes, which files, and how you will know it works (guide 3.2).



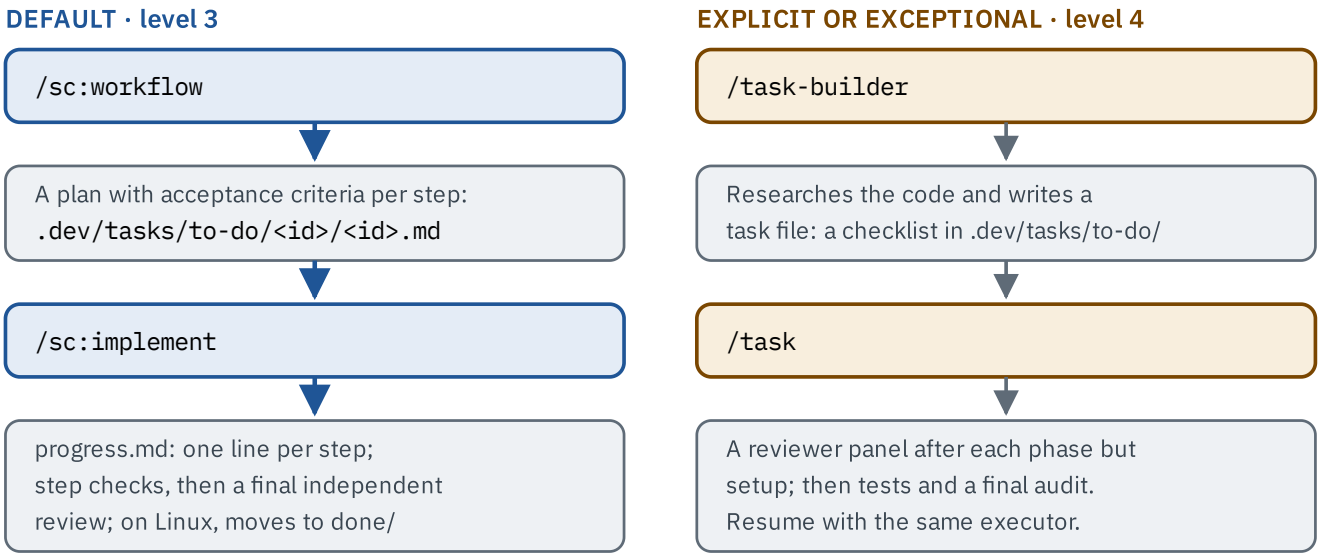
The six levels. Above the hard limit (which does not apply to contained `/sc:cleanup` and `/sc:improve` work on a folder), a written plan is normally level 3, not automatically level 4.

Level 4 is for an explicit choice of the task-file (MDTM) format, an existing MDTM task, or an exceptionally complex, long-running phased program that needs its research, checklist and phase-reviewer machinery. File count, sessions, an existing spec, PRD or TDD (product and design documents), risk or drift alone never make work level 4. After the level 6 documents, ordinary planning is level 3 (`/sc:workflow` , then `/sc:implement` when you ask for execution); `/roadmap` only if you explicitly choose it or have the exceptional level 4 need (guide 3.1).

Two planners

`/sc:workflow` with `/sc:implement` is the ordinary default for planned implementation. `/task-builder` with `/task` is for explicit or exceptional task-file (MDTM) work, level 4.

`/sc:workflow` writes a plan in which every step has acceptance criteria: a written definition of "done" for that step.



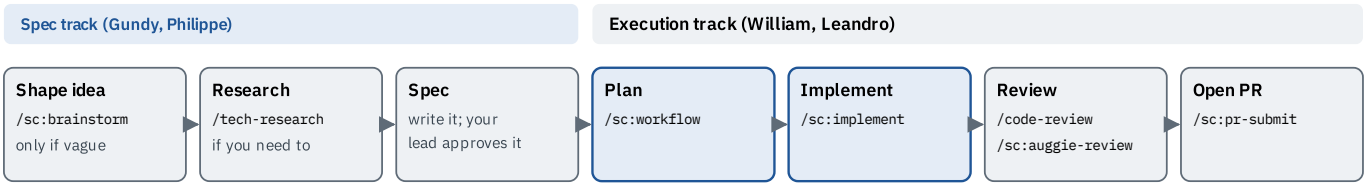
What each pair leaves on disk.

- **Run each plan with its own executor.** Folders named `TASK-WF-*` are `/sc:workflow` plans: run them with `/sc:implement`, never with `/task`. An existing MDTM task file goes directly to `/task`, without rebuilding it.

> *In the meeting `/sc:workflow` was described as "task builder light": faster and cheaper. It assumes you have done your homework, or lets you tell it plainly in a prompt what to plan. The rule of thumb given there: a plan made by an `sc:` command is followed up by an `sc:` command, and a plan from a plain slash command, for the most part, by a plain one. The guide's rule wins: when an approved plan already exists, use its own executor and do not rebuild it. A `/sc:workflow` package runs with `/sc:implement`; an existing MDTM task file runs with `/task`.*

From idea to merged PR

Most features follow the same path, from an idea to an open pull request (PR). Skip the steps you do not need: brainstorm only if the idea is vague, research only if you do not know how the current code works. Each later step runs only when you ask for it: a plan does not start the implementation, and implementing does not ship anything.



Level 6, a new product or major feature others build against: /prd, then /tdd, then Plan and Implement as usual; /roadmap only if explicitly chosen or exceptionally justified (guide 3.1). Explicit or exceptional task-file (MDTM) work swaps Plan and Implement for /task-builder and /task (guide 3.1 step 7).

The guide's feature path (3.2, with brainstorm from 3.1 and the review steps from 3.11), with the two tracks from the onboarding split overlaid.

Spec track (Gundy, Philippe)

Your work starts with a goal that still has to become a spec. These are the tools for that.

Shape a vague idea

`/sc:brainstorm` asks you questions, has several agents propose approaches, and merges them into requirements. It stops there; you decide the next step.

Find out what exists

`/tech-research` investigates *our* codebase. `/sc:research` searches the web. They look alike; they are not.

Check it is not already built

`/sc:recommend "<goal>" --plugin` searches the Claude plugin marketplaces and community skill repositories instead of the local commands.

> *The meeting's advice: before you build anything, look for someone who already built it, then fork it or use it.*

Get an expert critique

`/sc:spec-panel` has a panel of named requirements experts critique a spec. It only reports; you decide what to apply.

When are `/prd` and `/tdd` warranted? The full `/prd` then `/tdd` pipeline is level 6: a new product, a new component, or a major feature other people will build against. For a feature on an existing system, write a short spec and plan it with `/sc:workflow` (guide 3.2). An existing spec, many files or several sessions do not by themselves make it level 4; `/task-builder` and `/task` are only for explicit task-file (MDTM) work or an exceptionally complex, long-running program (guide 3.2 steps 3 and 5). If one part of that spec is a design other people will build against, the guide says: "Use `/tdd` when other people will build against the design" (guide section 5).

Execution track (William, Leandro)

- **Read the plan first.** Check that each step's acceptance criteria clearly say what "done" means.
- **Run it:** `/sc:implement .dev/tasks/to-do/<id>/<id>.md`. It builds one step at a time and checks each step's compliance with its acceptance criteria (small first-pass checks can be inline); the finished package then gets an independent final review.
- **The retry ladder:** when a step fails its check, it gets one fix pass, then up to three new attempts written from stronger models' advice. The step's acceptance criteria never change.
- **If interrupted,** run the same command again; it continues from the first unfinished step.

STOP CODE	WHAT IT MEANS	WHAT YOU DO
`E-HALT-QA`	A step still fails its check after the retries.	Fix the code and resume, or have your lead explicitly accept the explained gap.
`E-RESUME-NEEDED`	Something outside the step failed, such as the reviewer crashing.	Run the same command again.
`E-PLAN-CHANGED`	The plan was edited after the first step started.	Do not edit plans once a run has started.

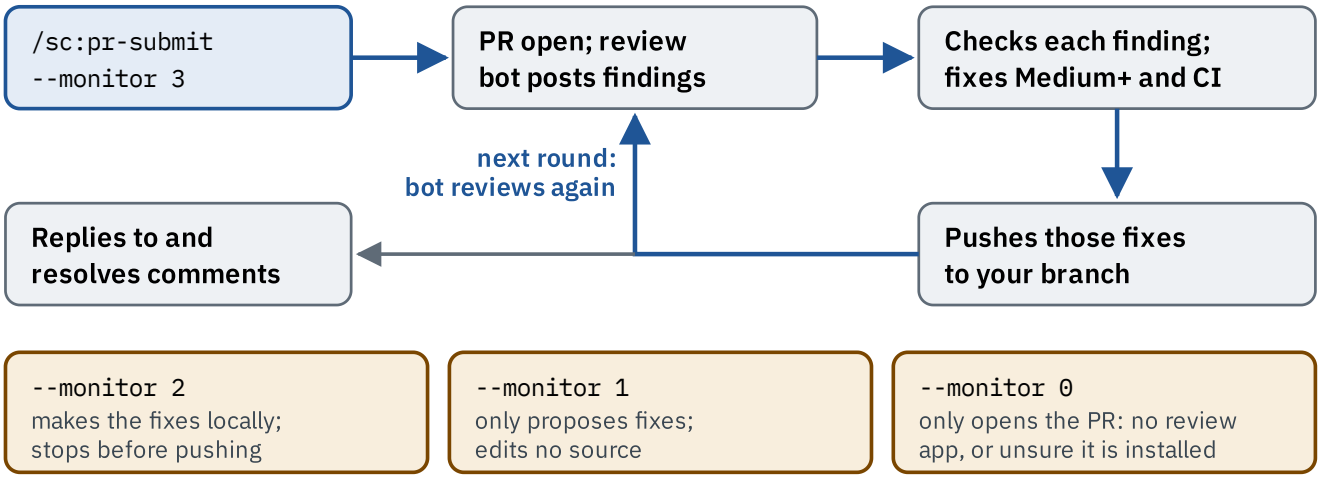
> *Why plans never change after step one, as the meeting put it: an agent that is allowed to edit its own success criteria will rewrite them so that it always succeeds.*

Review and ship

Review only: `/sc:auggie-review --no-post-pr --no-remediation-offer` on a PR, a diff or a snapshot keeps the findings local. Do not add commits, pushes or a PR to findings-only work.

Ship only when it is requested and authorized, in this order:

- **Commit locally** on your feature branch, only when authorized.
- **Bug review:** `/code-review`. **Security review** if you touched login, permissions, secrets, user input or file paths: `/security-review`. Authorize and record any fixes before the branch review.
- **Branch review:** `/sc:auggie-review --diff origin/<main branch>...HEAD --no-post-pr --no-remediation-offer`.
- **Push only with your lead's approval.**
- **Open the PR** with `/sc:pr-submit` and the one monitor ceiling you chose (below).



The PR loop at `--monitor 3`, the full loop, and the three lower ceilings. You choose exactly one and pass it explicitly.

Choose exactly one monitor ceiling for the job and pass it explicitly (guide 3.11): `--monitor 0` only opens the PR (no review app, or unsure it is installed); `--monitor 1` proposes fixes without editing source; `--monitor 2` makes the fixes locally and stops before pushing; `--monitor 3`, the command's current default, runs the full loop you authorized: it fixes confirmed review findings of Medium severity or higher (lower ones are only reported) and failing CI checks (the automatic tests and checks GitHub runs on the PR), pushes those fixes, and replies to and resolves the threads it fixed. The push rule (guide section 4): after your lead's approved push, `--monitor 3` pushes its own review fixes without asking again. If your lead wants to see those fixes first, use `--monitor 2`. A stronger project rule can require monitor 3; if that conflicts with a lower ceiling you were asked for, the guide says to disclose the conflict and not run it.

Where things live

Work is written to files

Plans, research and findings are saved to files, most under `.dev/`, not only into the chat. If a session dies, the files are what survives.

`superclaude update` overwrites

It overwrites every file it ships, including an edited `~/.claude/CLAUDE.md`, keeps no backup, and deletes and re-copies each shipped skill folder, so any file you added inside one is lost. Copy any file you changed or added before you run it.

Resume after a crash

Before you need it, tag the conversation with `/ccsession-tag <name>`; later, from a terminal in the same folder, run `ccsession <name>`.

This always works: run the same command again on the work's own files, `/sc:implement` on the plan or `/task` on the task file.

Not a resume method

`/sc:save` and `/sc:load` store and load Serena project memory (project notes). They do not resume a task or a plan.

Being worked out

Two topics came up in the onboarding meeting that the guide does not cover yet. Each has its own issue. Until they are settled, ask your lead.

Worktree and branch protocol

> **Worktree:** a second checkout of the same repository in its own folder, on its own branch, so parallel sessions do not touch each other's files.

When a session should get its own worktree and branch, where they live and how they are named is being worked out in issue #289.

Contributing rules

How team members propose or change skills, commands and agents, and what evidence a change needs, is being worked out in issue #290.

Traps

Look-alike commands (the guide explains each difference)

`/task`	`/sc:task`
`/sc:workflow` + `/sc:implement`	`/task-builder` + `/task`
`/sc:design`	`/tdd`
`/roadmap`	`/sc:roadmap`
`/tech-reference`	`/sc:document`
`/tech-research`	`/sc:research`
`/sc:cleanup-audit`	`/sc:cleanup`

Choosing level 4 for the wrong reason. The guide's rule: use `/task-builder` and `/task` only when you explicitly select the task-file (MDTM) format, or have an exceptionally complex, long-running phased program that needs their research, checklist and phase-reviewer machinery; an existing MDTM task goes straight to `/task`. File count, several sessions, an existing spec, risk, auditability, drift or resume alone never select them (guide sections 1 and 5).

> *The sign, as described in the meeting: a long run starts and many reviewer or quality-check (QA) helper agents, called subagents, start in the background without you asking for them.*

`/sc:save` is not a way to resume. Use the steps on slide 11.

When unsure

Ask for a recommendation

`/sc:recommend "<what you want to do, and why>"` picks the right command and writes a ready-to-paste prompt.

See what is installed

`/sc:help` lists the installed commands and skills and shows the six levels.

Ask your lead

Before starting anything large, or whenever this deck and the guide leave you unsure. If you work alone, read "your lead" as yourself.

Cheat sheet

LEVEL OR JOB	COMMANDS	GUIDE SECTION
1 · Ask a question	Ask in plain English, or `/sc:explain`	1, 3.5
2 · Make a small, direct change	`/sc:task`, or a plain edit, then run the test	1, 3.3
3 · Follow a checked implementation plan	`/sc:workflow`, then `/sc:implement` when you ask for execution	1, 3.2, 3.3
4 · Explicit or exceptional task-file work	`/task-builder` → `/task`; an existing MDTM task goes straight to `/task`	1, 3.1
5 · Research first	`/tech-research`; stop at the report, or plan with `/sc:workflow` if you decide to build	1, 3.5
6 · Write a full specification	`/prd` → `/tdd`; then `/sc:workflow` → `/sc:implement` if implementation is requested (`/roadmap` only if explicitly chosen or exceptionally justified)	1, 3.1
Fix a bug	`/sc:troubleshoot`	3.4
Review only, or ship	Review only: `/sc:auggie-review` with `--no-post-pr --no-remediation-offer`. Ship when requested and authorized: `/code-review`, `/sc:auggie-review`, then `/sc:pr-submit` with the one `--monitor` ceiling (0 to 3) you chose	3.11
Resume	The same command again, or `ccsession <name>`	3.12
Unsure	`/sc:recommend`, `/sc:help`, your lead	4

Full guide: HOW-WE-WORK.md on GitHub, installed at `~/.claude/HOW-WE-WORK.md`.

Built from HOW-WE-WORK.md at commit adaa28d9.

Source of truth for every command and flag: HOW-WE-WORK.md.